

NoSQL- Wide Column Store

Dr. Gerardo Rossel



DEPARTAMENTO
DE COMPUTACION

2025

Familia de Columnas o Wide Column

BigTable

BigTable - Google

Fay Chang, et al. **“BigTable: A Distributed Storage System for Structured Data.”** OSDI'06: Seventh Symposium on Operating System Design and Implementation, Seattle, WA, Nov., 2006.
<http://research.google.com/archive/bigtable.html>

BigTable - Definición

A Bigtable is a sparse, distributed, persistent multidimensional sorted map. The map is indexed by a row key, column key, and a timestamp; each value in the map is an uninterpreted array of bytes.

BigTable - Derivados

Hbase y Cassandra

Modelos de Datos Column Family

- HBase - BigTable.
- Cassandra.

ATENCION

Hay algunas diferencias de terminología y modelo general entre el modelo de HBase y el de Cassandra.

HBase

Conceptos

- **Namespace** es un agrupamiento lógico de tablas, similar a una base de datos en un SBDR.
- **Tablas**: Una tabla consiste de múltiples filas.
- **Row**: Una fila (row) se organiza como un conjunto de ***familias de columnas***. Las familias de columnas consisten de columnas relacionadas.
- **Cell**: Una combinación *row key*, familia de columnas y calificador de columna único identifica una celda (*cell*). La celda almacena datos que se denominan valores.
- **Timestamp**: Al escribir un nuevo valor el viejo no es sobrescrito sino que se agrega el nuevo junto a un *timestamp*. El *timestamp* permite a las aplicaciones determinar la última versión de un valor en una columna

Conceptos

Densidad

Densidad baja. En una fila, cada familia puede tener cualquier número de columnas, incluyendo la posibilidad de no tener ninguna.

Hash-Map

Básicamente una base column-family es un Mapa Multi-Dimensional

Modelo de datos

BigTable - HBase

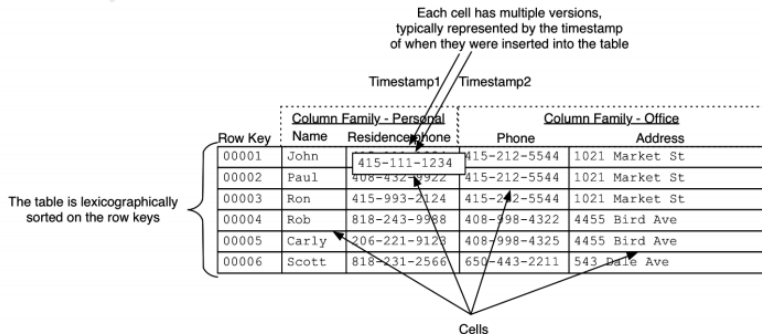


Figura de Amandeep Khurana: Introduction to HBase Schema Design

Relacional vs. Column Family

① Raw Data

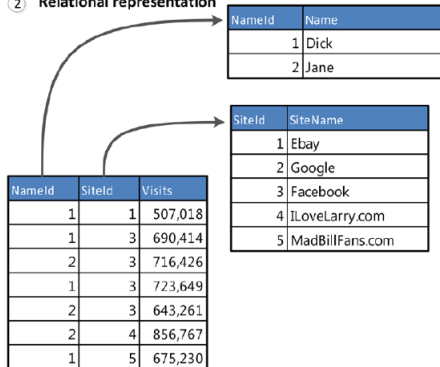
Name	Site	Visits
Dick	Ebay	507,018
Dick	Google	690,414
Jane	Google	716,426
Dick	Facebook	723,649
Jane	Facebook	643,261
Jane	ILoveLarry.com	856,767
Dick	MadBillFans.com	675,230

Relational vs. Column Family

① Raw Data

Name	Site	Visits
Dick	Ebay	507,018
Dick	Google	690,414
Jane	Google	716,426
Dick	Facebook	723,649
Jane	Facebook	643,261
Jane	ILoveLarry.com	856,767
Dick	MadBillFans.com	675,230

② Relational representation

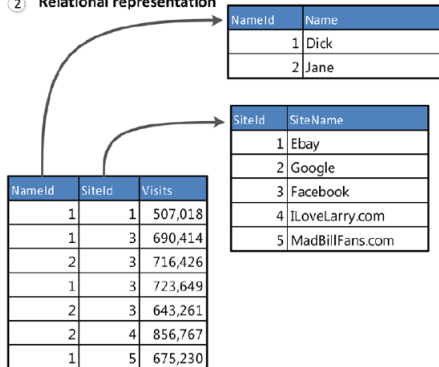


Relational vs. Column Family

① Raw Data

Name	Site	Visits
Dick	Ebay	507,018
Dick	Google	690,414
Jane	Google	716,426
Dick	Facebook	723,649
Jane	Facebook	643,261
Jane	ILoveLarry.com	856,767
Dick	MadBillFans.com	675,230

② Relational representation



③ HBase version

Id	Name	Ebay	Google	Facebook	(other columns)	MadBillFans.com
1	Dick	507,018	690,414	723,649		675,230

Id	Name	Google	Facebook	(other columns)	ILoveLarry.com
2	Jane	716,426	643,261		856,767

Cassandra

Cassandra Definición



Cassandra en 50 palabras

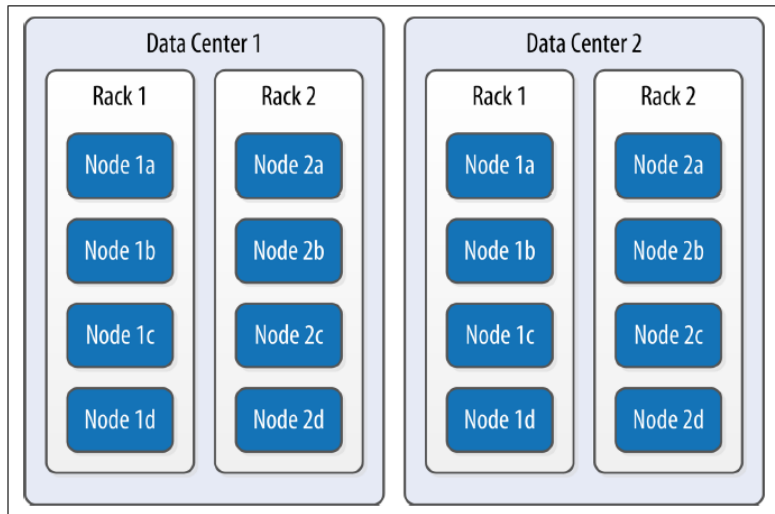
“Apache Cassandra is an open source, distributed, decentralized, elastically scalable, highly available, fault-tolerant, tuneably consistent, row-oriented database.

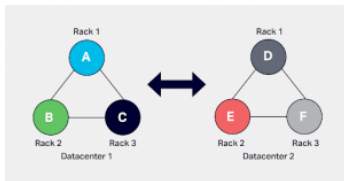
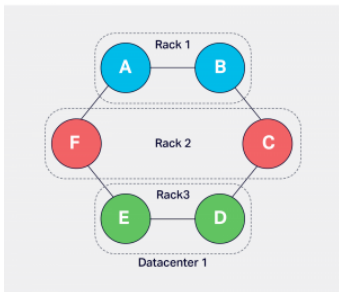
Cassandra bases its distribution design on Amazon's Dynamo and its data model on Google's Bigtable, with a query language similar to SQL. Created at Facebook, it now powers cloud-scale applications across many industries.”

Componentes

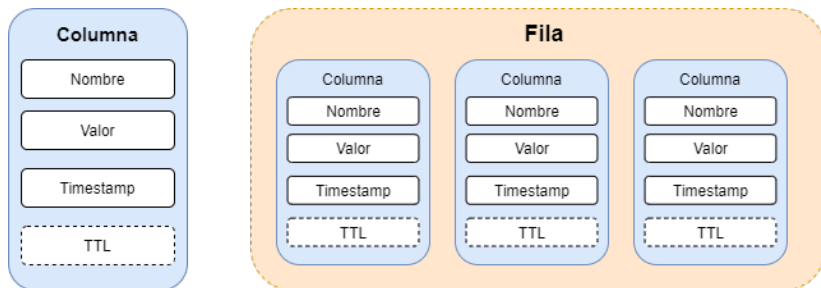
- **Node** Un servidor que ejecuta una instancia de Cassandra Un nodo puede ser un host físico, una instancia en la nube o incluso un contenedor Docker
- **Rack** Un conjunto lógico de nodos muy cerca uno de otro, quizás en máquinas físicas en un solo estante de equipo conectadas al mismo switch. En un *deployment* en la nube se refiere a una colección de instancias de maquinas corriendo en la misma zona de disponibilidad.
- **Data center** Un conjunto lógico de Racks quizás ubicado en el mismo edificio y conectado por red confiable. En la nube se mapea generalmente a una región de la misma.
- **Cluster:** el conjunto de data centers y que mapea a un anillo lógico

Organización física



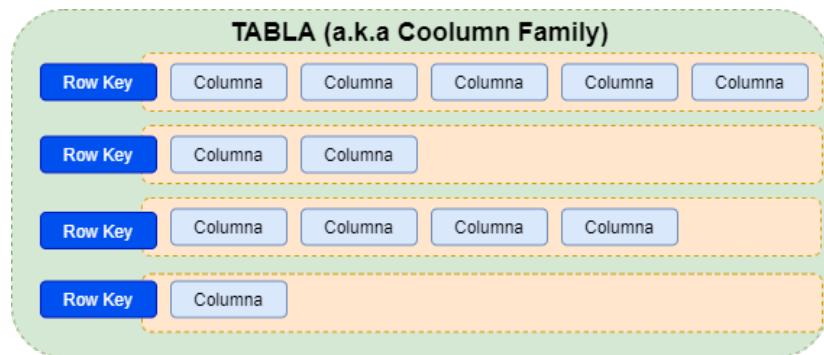


Modelo de Datos de Cassandra



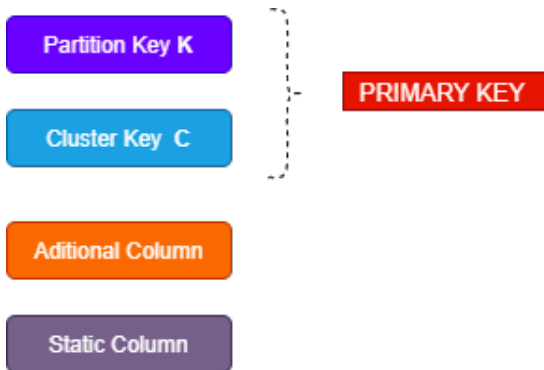
Filas y Columnas Cassandra

Modelo de Datos de Cassandra

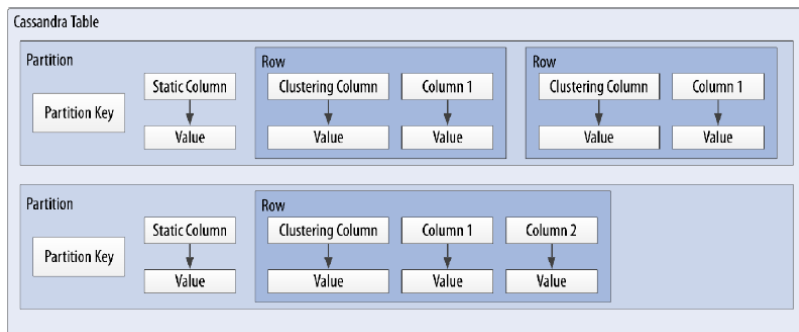


TABLAS Cassandra

Tipos de Columna



Tablas con Partición



Tomado de Cassandra: The Definitive Guide 2020

Ejemplo

Usuarios y Eventos

Una aplicación que rastrea usuarios y eventos. Se debería recordar cada vez que un cliente hace alguna interacción con el sistema: logins, compras, visitas a páginas, etc.

Ejemplo

Usuarios y Eventos

Una aplicación que rastrea usuarios y eventos. Se debería recordar cada vez que un cliente hace alguna interacción con el sistema: logins, compras, visitas a páginas, etc.

Solución relacional: dos tablas Usuarios y Eventos.

¿Que pasaría si hay 10.000.000 de usuarios y cada usuario tiene un promedio de 1000 eventos de 1Kb cada evento?

Ejemplo

Usuarios y Eventos

Una aplicación que rastrea usuarios y eventos. Se debería recordar cada vez que un cliente hace alguna interacción con el sistema: logins, compras, visitas a páginas, etc.

Solución relacional: dos tablas Usuarios y Eventos.

¿Que pasaría si hay 10.000.000 de usuarios y cada usuario tiene un promedio de 1000 eventos de 1Kb cada evento?

10TB de datos en la tabla de eventos.

Tabla en Cassandra

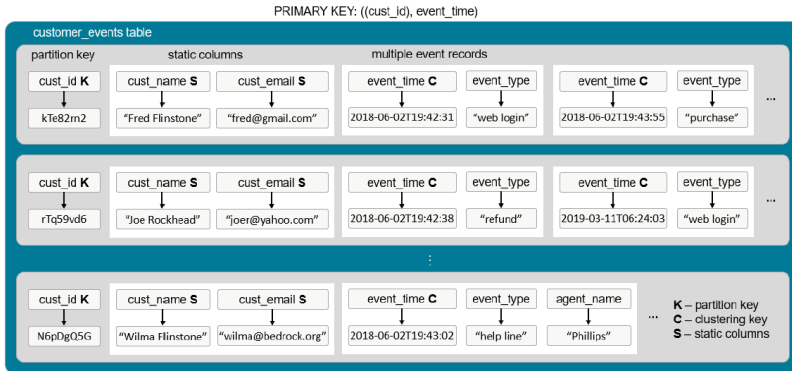
customer_events table

Primary Key

cust_id	cust_name	cust_email	event_time	event_type	agent_name	..
Aj50nT63	Barney Rubble	rubble@hotmail.com	2018-06-02T19:42:28	pageview	-	xxx
kTe82rn2	Fred Flinstone	fred@gmail.com	2018-06-02T19:42:31	web login	-	xxx
rTq59vd6	Joe Rockhead	joer@yahoo.com	2018-06-02T19:42:38	refund	-	xxx
N6pDgQ5G	Wilma Flinstone	wilma@bedrock.org	2018-06-02T19:43:02	help line	Phillips	xxx
kTe82rn2	Fred Flinstone	fred@gmail.com	2018-06-02T19:43:55	purchase	-	xxx
Aj50nT63	Barney Rubble	rubble@hotmail.com	2018-06-02T19:44:01	timeout	-	xxx
..			..			
rTq59vd6	Joe Rockhead	joer@yahoo.com	2019-03-11T06:24:03	web login	-	xxx

partition key static columns clustering key additional columns

Almacenamiento de la Tabla en Cassandra



Un poco de CQL

Tipos de Datos CQL

- Numéricos: bigint, smallint, tinyint, varint, float, double, decimal
- Texto: text, varchar (UTF-8 character string), ascii
- Tiempo: timestamp, date, time
- Identificadores: uuid, timeuuid
- Otros: boolean, blob, inet, **counter**
- Colecciones: set, list y map.
- Tuple. Ejemplo: address tuple<text, text, text, int>;
- User-Defined Types.

Crear una Tabla

```
CREATE TABLE IF NOT EXISTS rank_by_year_and_name (  
  race_year int,  
  race_name text,  
  cyclist_name text,  
  rank int,  
  PRIMARY KEY ((race_year, race_name), rank));
```

```
INSERT INTO cycling.rank_by_year_and_name  
  (race_year, race_name, cyclist_name, rank)  
VALUES (2015,  
  'Tour of Japan - Stage 4 - Minami > Shinshu',  
  'Benjamin PRADES', 1);
```

Restricción Clave de Partición

Operadores - Clave de Partición

Las columnas de la clave de partición solo admiten dos operadores: = e *IN*

Cassandra requerirá que restrinja todas las columnas de clave de partición o ninguna de ellas a menos que la consulta pueda usar un índice secundario.

Where en Clave de Partición

```
SELECT rank, cyclist_name as name
FROM cycling.rank_by_year_and_name
WHERE race_name = 'Tour of Japan - Stage 4 - Minami > Shinshu'
AND race_year = 2015
AND rank <= 2;
```

Where - Cluster Columns

```
CREATE TABLE numberOfRequests  
(cluster text,  
date text,  
datacenter text,  
hour int,  
minute int,  
numberOfRequests int,  
PRIMARY KEY ((cluster, date), datacenter, hour, minute))
```

Where - Cluster Columns

CORRECTO:

```
SELECT * FROM numberOfRequests  
WHERE cluster = 'cluster1' AND date = '2015-06-05'  
AND datacenter = 'US_WEST_COAST' AND hour = 14 AND minute = 00;
```

INCORRECTO:

```
SELECT * FROM numberOfRequests  
WHERE cluster = 'cluster1' AND date = '2015-06-05'  
AND hour = 14 AND minute = 00;
```


Where - Cluster Columns

Correctas:

```
SELECT * FROM numberOfRequests WHERE cluster = 'cluster1' AND  
      date = '2015-06-05' AND datacenter = 'US_WEST_COAST' AND  
      hour= 12 AND minute >= 0 AND minute <= 30;
```

```
SELECT * FROM numberOfRequests WHERE cluster = 'cluster1' AND  
      date = '2015-06-05' AND datacenter = 'US_WEST_COAST' AND  
      hour >= 12;
```

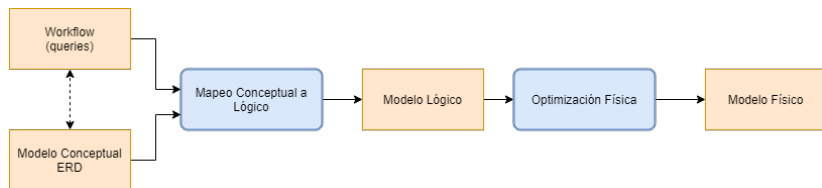
```
SELECT * FROM numberOfRequests WHERE cluster = 'cluster1' AND  
      date = '2015-06-05' AND datacenter > 'US';
```

Incorrecta:

```
SELECT * FROM numberOfRequests WHERE cluster = 'cluster1' AND  
      date = '2015-06-05' AND datacenter = 'US_WEST_COAST' AND  
      hour >= 12 AND minute = 0;
```

Diseño

Cassandra - Notación/Método Chebotko



- Modelo Conceptual (DER) y Workflow (consultas)
- Modelo Lógico
 - Buscar el subconjunto del modelo conceptual que satisface cada consulta
 - Elegir claves
 - Usar diagramas Chebotko para describir el modelo lógico

SE CREA UNA TABLA POR
CONSULTA

Reglas de Mapeo

Basados en los DMP (Data Modeling Principles), las reglas de mapeo ayudan a realizar la transición desde el modelo conceptual al modelo lógico.

- 1 **MR1** (*Entities and Relationships*): Los tipos de entidades y relaciones mapean a tablas mientras que los datos se asignan a filas. Los atributos de las entidades y las relaciones se mapean a columnas

Reglas de Mapeo

Basados en los DMP (Data Modeling Principles), las reglas de mapeo ayudan a realizar la transición desde el modelo conceptual al modelo lógico.

- 1 **MR1** (*Entities and Relationships*): Los tipos de entidades y relaciones mapean a tablas mientras que los datos se asignan a filas. Los atributos de las entidades y las relaciones se mapean a columnas
- 2 **MR2** (*Equality Search Attributes*): Si se utilizan en una consulta por igualdad de atributos, entonces, éstos se mapean a columnas del prefijo de la clave primaria. Dichas columnas deben incluir todas las columnas de clave de partición y, opcionalmente, una o más columnas clustering key.

Reglas de Mapeo

Basados en los DMP (Data Modeling Principles), las reglas de mapeo ayudan a realizar la transición desde el modelo conceptual al modelo lógico.

- 1 **MR1** (*Entities and Relationships*): Los tipos de entidades y relaciones mapean a tablas mientras que los datos se asignan a filas. Los atributos de las entidades y las relaciones se mapean a columnas
- 2 **MR2** (*Equality Search Attributes*): Si se utilizan en una consulta por igualdad de atributos, entonces, éstos se mapean a columnas del prefijo de la clave primaria. Dichas columnas deben incluir todas las columnas de clave de partición y, opcionalmente, una o más columnas clustering key.
- 3 **MR3** (*Inequality Search Attributes*): Si se utilizan en consultas por desigualdad, estos atributos mapean como columnas clustering key. En la definición de clave principal, una columna que participa en la búsqueda de desigualdad debe ubicarse después de las columnas que participan en la búsqueda de igualdad.

Reglas de mapeo. Continuación

- 4 **MR4** (*Ordering Attributes*): Mapea a una columna clustering key con orden ascendente o descendente según se especifique en la consulta
- 5 **MR5** (*Key Attributes*): Mapea a columnas en la clave primaria. Una tabla que almacena datos de entidades o relaciones como filas debe incluir atributos claves que identifique estos datos unívocamente

Cassandra - Notación/Método Chebotko

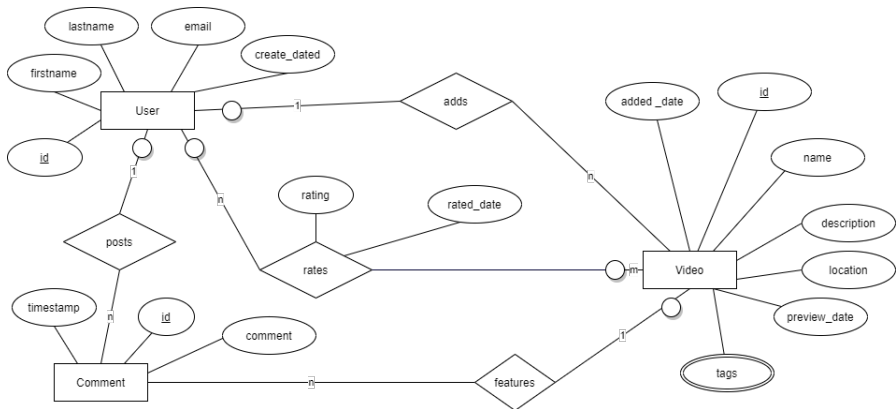
table_name	
column_name_1	CQL Type K ← Partition key column
column_name_2	CQL Type C↑ ← Clustering key column (ASC)
column_name_3	CQL Type C↓ ← Clustering key column (DESC)
column_name_4	CQL Type S ← Static column
column_name_5	CQL Type IDX ← Secondary index column
column_name_6	CQL Type ++ ← Counter column
[column_name_7]	CQL Type ← Collection column (list)
{column_name_8}	CQL Type ← Collection column (set)
<column_name_9>	CQL Type ← Collection column (map)
column_name_10	UDT Name ← UDT column
(column_name_11)	CQL Type ← Tuple column
column_name_12	CQL Type ← Regular column

Ejemplo

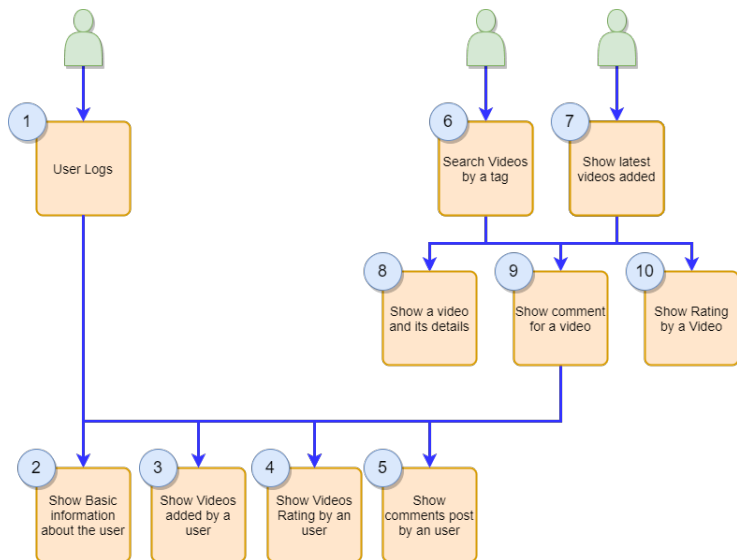
Aplicación de videos, los usuarios suben videos, califican los videos y comentan videos

Ejemplo

Aplicación de videos, los usuarios suben videos, califican los videos y comentan videos



Example Workflow



Identificar queries

Users



1

User logs into the site

Find a user by an email

2

Show basic information about a user

Find a user by an id

Comments



5

Show comments posted by a user

Find comments by a user (latest first)

9

Show comments for a video

Find comments by a video (latest first)

Videos



3

Show videos added by a user

Find videos by a user (latest first)

6

Search for videos by a tag

Find videos by a tag

7

Show latest videos added to the site

Find videos by a date added (latest first)

8

Show a video and its details

Find a video by an id

Ratings



4

Show video ratings by a user

Find ratings by a user (latest first)

10

Show ratings for a video

Find ratings by a user video (latest first)

Queries

- 1 user_by_email
- 2 user_by_id
- 3 videos_by_user
- 4 ratings_by_user
- 5 comments_by_user
- 6 videos_by_tag
- 7 latest_videos
- 8 videos_by_id
- 9 comments_by_video
- 10 ratings_by_video

Tablas x Query

1-User by email / 2-User by Id

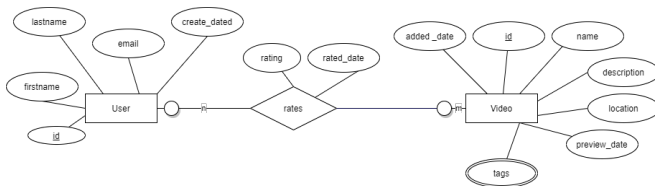
Tablas x Query

1-User by email / 2-User by Id

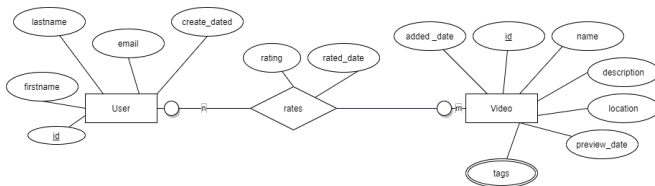
user_by_email	
email	K
userid	C↑
password	

users	
userid	K
firstname	S
lastname	S
email	S
created_date	S

10-Raitings by Video



10-Raitings by Video



ratings_by_video	
videoid	K
rated_date	C↓
userid	C↑
rating	
added_date	S
location	S
descripcion	S
preview_date	S
{tags}	S

Modelo Físico

ratings_by_video	
videoid	K
rated_date	C↓
userid	C↑
rating	
added_date	S
location	S
descripcion	S
preview_date	S
{tags}	S



Modelo Físico

ratings_by_video	
videoid	K
rated_date	C↓
userid	C↑
rating	
added_date	S
location	S
descripcion	S
preview_date	S
{tags}	S



ratings_by_video		
videoid	uuid	K
rated_date	timestamp	C↓
userid	uuid	C↑
rating	int	
added_date	date	S
location	text	S
descripcion	text	S
preview_date	text	S
{tags}	set<text>	S

CQL

```
CREATE TABLE raitnig_by_video(  
    videoid uuid,  
    rated_date,  
    userid uuid,  
    rating int ,  
    added date,  
    location text,  
    descripcion text,  
    preview_date text,  
    tags set<text> ,  
    PRIMARY KEY(videoid, rated_date, userid)  
) WHIT CLUSTERING ORDER BY (rated_date DESC, userid ASC)
```



Bibliografía

- Artem Chebotko, Andrey Kashlev, and Shiyong Lu. 2015. *A Big Data Modeling Methodology for Apache Cassandra*. In Proceedings of the 2015 IEEE International Congress on Big Data.
- Jeff Carpenter and Eben Hewitt. 2020. *Cassandra: The Definitive Guide* (3rd. ed.). O'Reilly Media, Inc.